Circuit-Aware Recipe Selection for Logic Synthesis A Supervised Learning Study

Shaan Chopra (240905140), Kenneth Rodrigues (240905112),
Darren Pais (240905093), and Aniket Verma (240905156)

School of Computer Science and Engineering

Manipal Institute of Technology

October 2026

Abstract

Logic synthesis transforms a digital circuit into an implementation with lower area or delay while preserving its input–output behavior. The order of the transformations affects the result, and a useful sequence for one circuit can perform poorly on another. We study supervised selection from a library of 120 transformation sequences. Our dataset contains 24,600 measured circuit–recipe pairs from 205 public benchmark circuits, generated with a fixed ABC synthesis tool and a NanGate 45 nm cell library. We describe circuits with graph statistics, baseline mapping measurements, and short diagnostic transformations; we encode recipes by their operations and their order. Models learn relative outcomes within each circuit. We evaluate on unseen circuits using repeated, nested, circuit-grouped cross-validation. Our primary selector gives 0.73% mean improvement in an equally weighted area–delay objective over the best fixed training recipe, with a 95% circuit-bootstrap interval of [0.25%, 1.18%]. Including feature collection, five learned candidates reach 3.95% mean regret in 3.11 s; eight globally ranked candidates reach 3.00% in 2.88 s. We also evaluate circuit-family transfer, feature ablations, and unseen recipes. The accompanying implementation includes the measured dataset, saved predictions, a trained selector, and commands for optimizing a new circuit.

1 Introduction

A digital circuit computes a function of binary inputs. An adder, for example, receives two binary numbers and produces their sum. Many different networks of logic gates can compute the same function. Their physical implementations can occupy different amounts of chip area and take different amounts of time to produce an output.

Logic synthesis searches for a useful implementation of that function. A synthesis tool can simplify expressions, share repeated computations, and restructure long chains of gates. These transformations preserve the function. Their order matters because one transformation changes the opportunities available to the next. A sequence of transformations is called a *recipe*.

Engineers often begin with a standard recipe that works reasonably well across designs. Another approach tries many recipes separately on each circuit. We study whether machine learning can use experience from earlier circuits to choose a useful recipe for a new one. The practical objective is to obtain a good implementation after a small number of complete synthesis runs.

We build a table of measured outcomes for 205 circuits and 120 recipes, train supervised models on circuit and recipe features, and evaluate the recipes those models select. The main comparison is with the single recipe that performs best on the training circuits. This comparison asks whether adapting the choice to each circuit provides value beyond learning a strong general-purpose recipe.

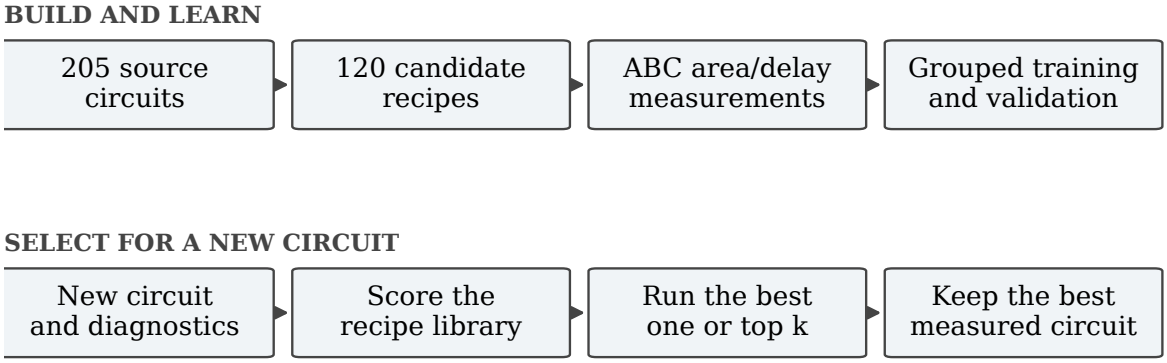


Figure 1: The experiment has a dataset-building stage and a selection stage. During evaluation, the new circuit belongs to an outer test fold and contributes no labels to training or tuning.

2 Circuit Concepts and the Synthesis Process

2.1 Combinational circuits, area, and delay

Our circuits are *combinational*: their outputs depend on their current inputs. They contain no memory elements such as registers or latches. This gives each circuit a fixed Boolean input–output function.

Area is the sum of the areas of the logic cells used to implement the circuit, measured here in square micrometres (μm^2). Lower area allows the logic to occupy less chip space. *Delay* is the estimated time along the slowest input-to-output path, measured in picoseconds (ps). Lower delay allows an output to become valid sooner and can help a design meet its timing requirement. Area and delay can trade off: a larger cell may drive a signal faster, while sharing logic can reduce area and add a longer path.

ABC estimates these quantities after *technology mapping*: assigning the logic to small predesigned gates, called standard cells, from a technology library. The library is a catalogue of available cell functions, areas, and timing models. We use the NanGate Open Cell Library for a nominal 45 nm technology at its typical operating corner, meaning its nominal voltage and temperature condition [5]. The resulting *netlist* lists the cells and their connections. Our measurements come from synthesis and cell timing with no wire-load model. Placement and routing would add physical interconnect effects.

2.2 And-Inverter Graphs

ABC represents the circuit internally as an *And-Inverter Graph* (AIG) [1, 2]. Each internal node computes AND of two signals; an edge can invert its signal. This representation can express

arbitrary Boolean logic. For example, OR can be expressed using AND and inversion:

$$a \vee b = \neg(\neg a \wedge \neg b).$$

Input nodes supply signals, output connections identify the results, and shared graph nodes represent shared computations. The graph is directed and has no cycles for a combinational circuit.

Two simple graph measurements are the number of AND nodes and the number of *levels*: the largest number of AND nodes on an input-to-output path. Node count summarizes logic size; levels summarize logical depth. They are useful structural features. The final cell area and timing also depend on how groups of nodes map to the library's cells.

ABC's `strash` command constructs a structurally hashed AIG, merging identical AND nodes. We use this graph as the starting point for every recipe on a circuit.

2.3 What a recipe does

Our vocabulary contains seven operations, listed in Table 1. Each operation visits and transforms the current graph. A recipe applies its operations sequentially to that graph. The next operation therefore receives the previous operation's result.

Symbol	ABC command	Purpose
`rw`, `rwz`	`rewrite`, `rewrite -z`	Replace small parts of the graph with equivalent structures, using existing shared logic where possible.
`rf`, `rfz`	`refactor`, `refactor -z`	Re-express a larger logic region through factoring. For example, repeated terms in $ab + ac$ suggest the factored form $a(b + c)$.
`rs`, `rsz`	`resub`, `resub -z`	Express a function using signals already computed elsewhere in the circuit.
`b`	`balance`	Restructure chains of associative logic into a shallower arrangement.

Table 1: Recipe operations. The `-z` variants also allow replacements with no immediate reduction in graph size. Such a replacement can create opportunities for a later operation. It still takes computation time.

The standard recipe `resyn2` interleaves ten passes [2]:

`b rw rf b rw rwz b rfz rwz b`.

For example, it first balances the graph, then rewrites it, then refactors it. We include this explicit sequence as recipe R001 and use it as a reference throughout the experiment. The model selects a complete sequence from the candidate library.

2.4 One complete measurement

For each circuit–recipe pair, we start a fresh ABC process and execute:

```
read_lib -w NangateOpenCellLibrary_typical.lib
read_blif circuit.blif       # read_bench for a .bench input
strash
balance; rewrite; ...       # all operations in one recipe
```

```
print_stats
map
topo
stime
```

The input file describes a network of Boolean logic. BLIF and BENCH are text formats that ABC can read directly. After the recipe finishes, `map` assigns the resulting logic to cells from the library. The `topo` command orders the mapped network for timing analysis, and `stime` reports area and delay. We map once, after the complete recipe.

Every candidate recipe starts again from the original circuit and its hashed AIG. Recipe R002 therefore does not inherit the graph produced by R001. The baseline measurement follows the same process with the recipe line omitted. This gives each circuit its own baseline area A_0 and delay D_0.

3 Dataset Construction

3.1 Circuit sources and inventory

We retain the 205-circuit inventory from our pilot study. We retrieve EPFL inputs from the EPFL benchmark repository [3]; ISCAS-85 and MCNC inputs come from the ISP RAS `hdl-benchmarks` repository [4]. MCNC and LGSynth'91 originate from the 1991 logic-synthesis benchmark collections. Our selected files are from the MCNC combinational BLIF directory. The distribution is shown in Table 2.

Family	Circuits	Min. nodes	Median	Max. nodes	Depth median
EPFL arithmetic	7	1,020	5,416	27,062	255
EPFL control	9	174	693	13,758	20
ISCAS-85	10	209	610	2,337	30
MCNC	179	40	270	7,803	14

Table 2: Circuit inventory. Size and depth refer to the starting hashed AIG. The four families are also used for validation across circuit types.

EPFL contains arithmetic designs such as adders and multipliers, together with controllers, encoders, and related logic. ISCAS-85 provides established gate-level test circuits. MCNC contributes many smaller logic networks and supplies most of the independent training examples.

The pilot inventory retains circuits with no latches, at least four primary inputs, and at least 40 AND nodes. Its size limits are 30,000 AND nodes for EPFL and 8,000 for MCNC. ISCAS-85 circuits fit below 2,400 nodes. The inventory excludes the ISCAS-85 copies distributed inside MCNC and removes three duplicate signatures based on input count, output count, node count, and depth. These four counts are a coarse screening signature; different functions can share them. We retain the resulting inventory consistently across our experiments and save a source-file checksum for every circuit.

There are 179 MCNC circuits and 26 circuits from the other three families. We therefore report family-specific results and explicitly test transfer between families. The largest circuit has 27,062 AND nodes. The inventory concentrates on benchmark-scale combinational logic.

3.2 The 120-recipe library

Each recipe contains 6–12 operations. We preserve the pilot's 60 sequences and add 60 new sequences with random seed 17. The final library contains two established scripts, `resyn` and `resyn2`, plus 24 area-weighted, 24 delay-weighted, 24 mixed, and 46 uniformly sampled recipes.

The labels describe sampling distributions. Area-weighted sampling favors rewriting, refactoring, and resubstitution; delay-weighted sampling favors balancing. Mixed sampling gives balancing a modest extra weight. Length is sampled uniformly, duplicate sequences are rejected, and consecutive balancing operations are rejected. Each sequence is evaluated to establish its actual performance. Its sampling label is excluded from the model inputs.

3.3 Labels and objective

The completed dataset contains one row for each of the $205 \times 120 = 24,600$ circuit–recipe experiments. Each row stores the circuit family and identifier, starting-circuit features, recipe operations, baseline measurements, post-recipe measurements, and runtime. All 24,600 runs completed successfully. The same 120 candidates are available for every circuit.

We normalize the mapped measurements by that circuit's baseline:

$$y_A(C,R) = \frac{A(C,R)}{A_0(C)}, \qquad y_D(C,R) = \frac{D(C,R)}{D_0(C)}. \tag{1}$$

A value of 0.90 means a 10% reduction from the corresponding baseline. This normalization lets us compare improvements across circuits of different sizes.

We combine the two outcomes using a weight λ:

$$J_\lambda(C,R) = \lambda y_A(C,R) + (1-\lambda) y_D(C,R). \tag{2}$$

Lower is better. Our primary experiment uses $\lambda = 0.5$, giving equal weight to relative area and delay. Separate experiments use $\lambda = 0$ for delay and $\lambda = 1$ for area. A score of 0.90 is a 10% reduction in this weighted normalized objective; the individual area and delay changes are reported separately when interpreting a selected implementation.

Recipe	**Area (μm^2)**	**Delay (ps)**	y_A	y_D	$J_{0.5}$
R001 (resyn2)	385.97	621.87	1.083	0.929	1.006
R037	343.94	602.63	0.965	0.900	0.932

Table 3: Two measured recipes for circuit `c1908`. Its baseline area is 356.44 μm^2 and baseline delay is 669.63 ps. R037 gives lower area and delay than `resyn2` on this circuit.

4 Features and Learning

4.1 Describing the circuit

We extract 50 circuit features in three groups. Their definitions and reasons for inclusion are summarized below; Appendix A gives the complete structural list.

Static graph statistics (24). Input and output counts describe the interface. Node count and depth describe size and path length. *Fanout*, the number of connections using a signal, describes sharing and branching. The number of nodes at each level describes whether the graph is wide

or concentrated along long paths. Differences between the levels of an AND node's two inputs indicate uneven paths that balancing may change. Additional statistics measure complemented edges, small exclusive-OR patterns, and nearby paths that split and meet again. We parse these quantities from ABC's binary AIGER graph export. We apply $\log(1+x)$ to ten size-like features so that large circuits have a more moderate numerical scale.

Mapping baseline features (4). We add $\log A_0$, $\log D_0$, area per AND node, and delay per graph level. These measurements describe how the starting graph maps to the chosen cell library. They are available before a candidate recipe is selected.

Probe responses (22). A probe is a short diagnostic transformation. Starting from the original hashed graph, we apply each of the seven operations once and record the ratios of its resulting node count and depth to the starting values. This gives 14 features. We also apply four two-step probes: `b rw`, `rw b`, `rf b`, and `b rf`. Their node and depth ratios give eight more features. These pairs expose order sensitivity. Each probe starts independently from the same graph; probes never accumulate into a longer optimization chain. They stop before mapping, so their outputs are graph measurements. The complete probe set takes 0.91 seconds per circuit on average in our sweep environment.

4.2 Describing the recipe

The recipe encoding has 144 features: length and count of `-z` operations (2), counts of the seven operations (7), one-hot operation identities at 12 positions (84), counts of the 49 ordered adjacent operation pairs, and indicators for a balancing operation at the beginning and end (2). Short sequences use zero padding at unused positions.

Operation counts describe the mixture of transformations. Position and transition features preserve order: `rw b` and `b rw` have identical operation counts but different position and transition values. The full circuit–recipe input has $50 + 144 = 194$ features. Identifiers, measured post-recipe area and delay, post-recipe graph statistics, and runtimes are excluded.

4.3 Predicting relative performance

The selector needs a useful ordering of recipes for the same circuit. Some circuits improve substantially under almost any recipe; others offer little improvement. To focus the model on differences between recipes, we center the training objective within each training circuit:

$$\widetilde{J}(C,R) = J(C,R) - \frac{1}{|\mathcal{R}_{\text{train}}|} \sum_{R' \in \mathcal{R}_{\text{train}}} J(C,R'). \tag{3}$$

A negative target means that a recipe is better than that circuit's average candidate. At prediction time, the model ranks recipes using the predicted centered values. All candidates for a circuit share the same missing offset, so this ranking requires no test-circuit outcome labels.

We compare five model variants: additive ridge regression, ridge with explicit interactions, Extra-Trees, Random Forest, and gradient-boosted decision trees implemented with LightGBM. The interaction ridge model multiplies each of the 22 probe features by each of the seven operation counts, adding 154 columns. The ordinary ridge model adds separate circuit and recipe contributions. Its predicted recipe ordering is therefore the same for every circuit. The interaction features allow its ordering to depend on the circuit; the tree models learn interactions through their splits.

We standardize features for ridge using statistics fitted on the training partition. We also

compare raw J targets and within-circuit ranks scaled to $[0, 1]$ in the target ablation. Every target transformation uses only the training rows available to that fit.

4.4 Turning predictions into a decision

Our primary selector scores all 120 library recipes and chooses the lowest prediction. We evaluate each model separately and an *adaptive selector* that chooses its model family and settings using inner validation on the training circuits.

We also evaluate two extensions. A *margin gate* uses the predicted best recipe when its predicted advantage over the best fixed recipe exceeds a threshold; otherwise it uses that fixed recipe. We tune the threshold inside the training partition. A *top-k selector* executes the k lowest-scored recipes, measures their actual mapped outcomes, and keeps the best. We use $k \in \{1, 2, 3, 5, 8\}$. The shortlist ranking is determined entirely by the model; the final choice among those executed candidates uses their measurements.

5 Validation and Comparisons

5.1 Unseen circuits and nested tuning

The independent examples are the 205 circuits. The 24,600 rows contain repeated observations of those circuits. We keep every recipe row belonging to a circuit in the same partition.

Our primary protocol uses five circuit folds repeated three times with seeds 0, 1, and 2. Families are stratified where the available counts permit. In each outer fold, approximately 164 circuits are used for training and 41 are held out for testing. Every circuit is tested once per repeat.

Inside an outer training partition, three-fold circuit-grouped validation selects model settings. Its criterion is mean percentage improvement over the best fixed recipe learned from the corresponding inner training circuits. The adaptive selector also chooses its model family using these inner results. We then refit on the complete outer training partition and evaluate its recipe choices on the outer test circuits. This separates training, tuning, and testing at the circuit level. Table 4 gives the fixed tuning grid.

The exhaustive dataset supplies the outcome of every candidate. We evaluate a selector by first making its feature-based decision and then looking up the measured outcome of its chosen recipe. A top-k selector can use the measurements of its k recommended candidates to choose its final result. The complete test-circuit outcome table is used to compute the evaluation metrics and oracle reference.

Model	**Settings and inner-validation choices**
Ridge, interaction ridge	Regularization $\alpha \in \{10, 100\}$; training-fitted standardization.
Extra-Trees, Random Forest	80 trees; maximum depth 16; 30% of features considered per split; minimum leaf size in $\{5, 15\}$.
LightGBM	180 trees; learning rate 0.05; minimum leaf size 20; L_2 penalty 1; number of leaves in $\{15, 31\}$.
Margin gate	Predicted advantage threshold in $\{0, 0.0025, 0.005, 0.01, 0.02\}$.

Table 4: Model settings. The main comparison uses centered targets and all 194 features; interaction ridge additionally uses the 154 explicit products.

5.2 Reference selectors

Best fixed recipe. We average each recipe's measured objective across the training circuits and choose the smallest average. That single recipe is then used on every test circuit. We recompute it separately inside every training split. It is the primary comparator.

`resyn2`. We execute the standard ten-operation script described in Section 2.3. This is a familiar synthesis reference and is distinct from the no-recipe normalization baseline.

Random choice. We report the expected result of uniform recipe selection. For a budget of k runs, we compute the exact expected best outcome among k distinct uniformly sampled recipes.

Global top-k. We execute the k recipes with the best training-set mean objectives and keep their best measured result on each test circuit. This is the budget-matched comparator for a learned shortlist.

Oracle. We take the best measured recipe in the entire candidate library for each circuit. This summarizes the improvement available within the library. It requires the exhaustive measurements of all 120 recipes.

5.3 Metrics, uncertainty, and execution cost

For each test circuit, we compute percentage improvement over the fixed training recipe:

$$\text{Improvement}(C) = 100\,\frac{J_{\text{fixed}}(C) - J_{\text{selected}}(C)}{J_{\text{fixed}}(C)}. \tag{4}$$

Positive values favor the selector. Our primary endpoint is the mean of this quantity for the adaptive one-recipe selector at $\lambda = 0.5$. We also report median improvement, win and loss rates, improvement over `resyn2`, and regret relative to the oracle:

$$\text{Regret}(C) = 100\,\frac{J_{\text{selected}}(C) - J_{\text{oracle}}(C)}{J_{\text{oracle}}(C)}. \tag{5}$$

Regret is zero when the selected outcome reaches the best measured library outcome. Spearman correlation measures within-circuit ranking quality; we assign zero when predictions or measured outcomes are constant within a circuit.

We average repeated-test improvements for each circuit, then bootstrap those 205 circuit averages with 2,000 resamples to obtain 95% intervals. Each circuit receives equal weight. A paired Wilcoxon signed-rank test uses the same per-circuit averages for the primary comparison. Model-specific and ablation results describe secondary comparisons.

Execution cost includes the learned selector's graph extraction, baseline mapping, probes, prediction, and all k verified recipe runs. We time prediction with the fitted model already loaded. Global and random selectors pay for their executed candidates. We report the measured time and express it in run equivalents by dividing by the dataset's mean complete-recipe runtime. Synthesis times were recorded during a five-worker CPU sweep, so they describe that execution setting. Candidate-count and time plots show the cost of obtaining a better shortlist outcome.

5.4 Transfer, ablations, and unseen sequences

We train on the 179 MCNC circuits and test on the 26 EPFL and ISCAS-85 circuits to examine transfer to a different mixture of designs. We also hold out each of the four families in turn.

The feature ablation compares static graph statistics, static plus mapping measurements, all features, and probes alone. The recipe-encoding ablation adds positions and then transitions

to operation counts. Target ablations compare raw, centered, and ranked objectives. Library experiments use 30, 60, and 120 candidates. We use the boosting model and one five-fold repeat for these controlled comparisons, tuning its settings inside each training fold. The 60-recipe experiment uses the original pilot library; the 30-recipe subset retains both established scripts and samples the other strata with a fixed seed.

We also repeat the boosting experiment with 25, 50, and 100 randomly sampled training circuits per outer fold. The outer test circuits stay fixed, and the same circuit-count limit applies before inner tuning. This measures how selection quality changes with the number of independent training designs.

For the unseen-recipe experiment, we reserve 24 sequences, stratified across the sampled recipe groups. Their labels are removed from all training and inner-validation partitions. On held-out circuits, the model scores the full 120-recipe library, including the reserved sequences. This tests whether the sequence encoding supports selection when some candidate recipes have no training labels. Both established scripts remain available during training.

6 Results

6.1 Measured dataset and selection headroom

The full sweep completed in 30.7 minutes using five synthesis workers. The median circuit has 327 AND nodes and depth 16. Across all measured pairs, the mean normalized area is 0.834 and the mean normalized delay is 0.875. These dataset averages summarize the candidate library's behavior before learning.

The best recipe varies across circuits. With ties resolved by recipe identifier, the oracle selects 85 different sequences across the pool at $\lambda = 0.5$. Selecting each circuit's best measured recipe gives 9.15% mean improvement over the leave-one-circuit-out best fixed recipe. This establishes the library's circuit-specific selection headroom.

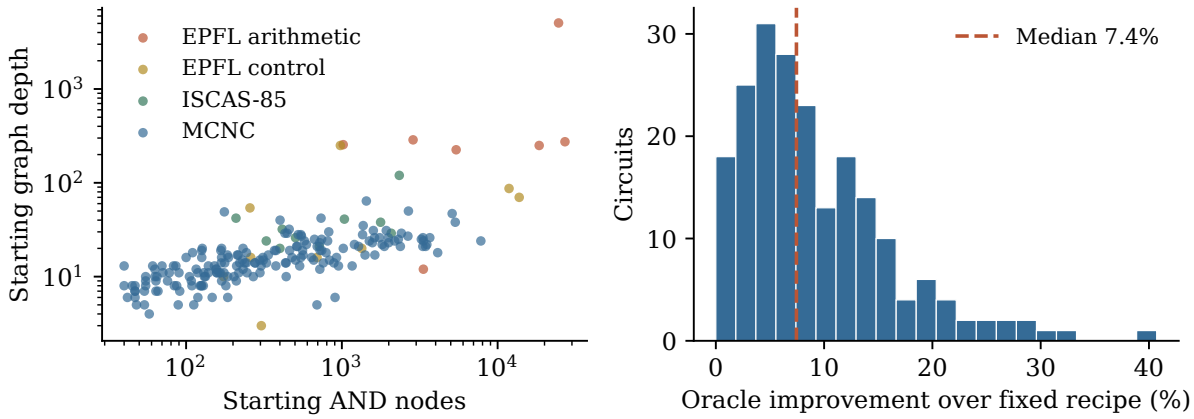


Figure 2: Circuit pool and available selection headroom. Left: starting AIG size and depth, colored by source family. Right: improvement of each circuit's best measured recipe over a fixed recipe chosen from the other 204 circuits, at $\lambda = 0.5$.

6.2 Selection on held-out circuits

Table 5 reports the main repeated-validation results. All learned selectors use one complete recipe execution after feature collection. The adaptive selector's model family is selected inside the training partition. Its mean improvement is 0.73% with interval [0.25%, 1.18%]; its mean regret is 9.01%. The paired test for the primary comparison gives $p = 2.82 \times 10^{-4}$.

Selector	Gain (%)	95% interval	Regret (%)	Win (%)
Additive ridge	0.12	[-0.06, 0.31]	9.81	20.5
Interaction ridge	0.02	[-0.75, 0.78]	9.74	47.8
Extra-Trees	1.11	[0.60, 1.62]	8.55	63.4
Random Forest	0.68	[0.04, 1.28]	9.06	56.6
Gradient boosting	0.45	[-0.18, 1.02]	9.29	56.6
Adaptive selector	0.73	[0.25, 1.18]	9.01	58.5
Adaptive + margin gate	0.72	[0.24, 1.16]	9.04	56.6
Uniform random (expected)	-1.94	[-2.53, -1.32]	11.83	–
resyn2	-0.02	[-0.94, 0.86]	9.66	51.2
Best fixed training recipe	0.00	[0.00, 0.00]	9.97	0.0
Oracle (120 measured runs)	8.61	[7.90, 9.39]	0.00	98.5

Table 5: One-recipe selection at $\lambda = 0.5$. Improvements are relative to the best fixed training recipe. Intervals bootstrap the 205 circuit averages after combining the three repeats. Win rates use those per-circuit averaged improvements.

The adaptive selector chose Random Forest in 7, Gradient boosting in 2, Additive ridge in 2, Interaction ridge in 2, Extra-Trees in 2 of the 15 outer folds. Its average improvement over `resyn2` was 0.36%. After averaging the three test results for each circuit, it improved the fixed-recipe objective on 58.54% of circuits and increased it on 35.61%. Its mean within-circuit Spearman correlation was 0.268. The margin gate produced a mean gain of 0.72% with interval [0.24%, 1.16%]. The confidence interval lies above zero, supporting an average gain on this circuit pool.

Selector	Area gain (%)	Delay gain (%)	Both improve (%)
Adaptive, one recipe	0.54	0.54	26.8
Adaptive, five recipes	2.21	7.53	67.3
Global, five recipes	1.75	7.81	65.4
resyn2	-2.63	1.81	18.0

Table 6: Individual area and delay improvements over the best fixed training recipe at $\lambda = 0.5$. Positive gains mean reductions. Each metric is averaged within a circuit across repeats and then across the 205 circuits.

For the one-recipe adaptive selector, mean area improvement was 0.54% and mean delay improvement was 0.54% relative to the fixed recipe. Both improved on 26.83% of circuits after combining repeats. With five verified recipes, the corresponding area and delay improvements were 2.21% and 7.53%. The weighted objective permits an area–delay trade-off, so these component measurements help interpret the combined gain.

6.3 Shortlists and their cost

Runs k	Learned (%)	Global (%)	Random (%)	Learned (s)	Global (s)
1	9.01	9.97	11.83	1.66	0.37
2	6.47	7.01	8.45	2.03	0.73
3	5.41	5.68	6.93	2.39	1.09
5	3.95	4.13	5.36	3.11	1.81
8	3.06	3.00	4.17	4.20	2.88

Table 7: Mean oracle regret for the adaptive learned shortlist and its reference selectors. Every method executes k complete recipes; learned selection additionally collects circuit features. Lower regret is better.

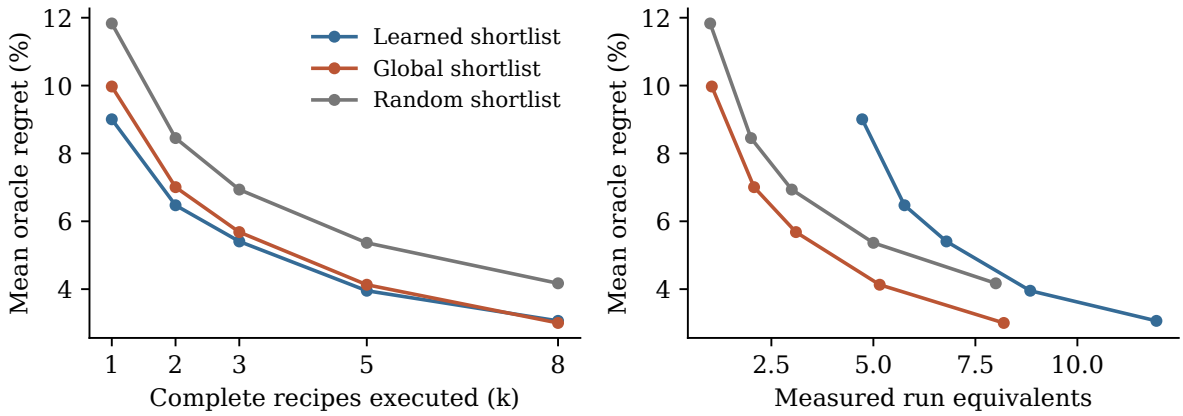


Figure 3: Shortlist quality versus executed candidates and measured run equivalents. The time axis includes baseline mapping, graph extraction, probes, prediction, and verification for the learned selector. Global and random shortlists use their executed-recipe times.

Increasing the learned shortlist from one to five recipes changed mean regret from 9.01% to 3.95%. At five executions, the global shortlist reached 4.13% regret. The learned five-recipe selector took 3.11 seconds per circuit on average, including feature collection; the global five-recipe selector took 1.81 seconds. The one-recipe learned selector used 4.73 mean-run equivalents. The closest tested global budget to the one-recipe learned selector was $k = 5$, with 1.81 seconds of execution time and 4.13% mean regret. The global eight-recipe shortlist reached 3.00% regret in 2.88 seconds, compared with 3.95% in 3.11 seconds for five learned candidates. In this cost estimate, executing more globally ranked candidates gave a better outcome than collecting the full feature set for the learned shortlist.

6.4 Which information helps?

Setting	Features	Gain (%)	Regret (%)
Full features; centered; full encoding	194	0.81	8.86
Static graph features	168	0.00	9.76
Static + mapping features	172	0.21	9.47
Probe features alone	166	0.51	9.21
Raw objective target	194	0.77	8.87
Within-circuit rank target	194	1.13	8.42
Operation-count encoding	59	0.55	9.10
Counts + position encoding	143	0.29	9.42

Table 8: Controlled boosting-model ablations at $\lambda = 0.5$, evaluated on the same first five outer folds. Full features, full recipe encoding, and centered targets form the reference setting. Each row changes the stated component.

The full boosting configuration achieved 0.81% mean gain in this repeat. Static graph features gave 0.00%; adding mapping measurements gave 0.21%; probe features alone gave 0.51%. Using raw targets produced 0.77% gain, and rank targets produced 1.13%. Operation counts alone gave 0.55% gain; the full sequence encoding gave 0.81%.

6.5 Transfer and candidate-library changes

Test distribution	Circuits	Gain (%)	95% interval	Regret (%)
MCNC to EPFL + ISCAS-85	26	1.61	[-0.06, 3.31]	8.75
EPFL arithmetic held out	7	0.95	[-2.49, 4.91]	8.72
EPFL control held out	9	-2.69	[-9.61, 3.16]	15.77
ISCAS-85 held out	10	0.98	[-0.35, 2.38]	8.72
MCNC held out	179	0.00	[0.00, 0.00]	10.34

Table 9: Adaptive one-recipe selection under changes in circuit distribution. Training and tuning use only circuits outside the stated test family or set. The MCNC-to-other split tests all 26 EPFL and ISCAS-85 circuits together.

Training on MCNC and testing on the other 26 circuits gave 1.61% mean gain, with interval [-0.06%, 3.31%]. In the primary repeated protocol, family-specific mean gains were EPFL arithmetic 0.39%; EPFL control 0.63%; ISCAS-85 1.56%; MCNC 0.70%. The held-out arithmetic and control families contain seven and nine circuits, respectively, which gives their interval estimates a small sample base.

Experiment	Gain (%)	95% interval	Regret (%)
Delay objective	-0.79	[-2.27, 0.70]	18.61
Area objective	-0.28	[-1.05, 0.48]	7.41
30-recipe library	0.36	[-0.45, 1.15]	6.91
60-recipe library	0.37	[-0.59, 1.30]	8.05
25 training circuits	0.19	[-0.65, 1.09]	9.30
50 training circuits	0.13	[-0.73, 0.95]	9.41
100 training circuits	-0.62	[-1.69, 0.37]	9.69
24 recipes unseen in training	0.66	[-0.28, 1.56]	8.95

Table 10: Boosting-model sensitivity experiments. Each objective weight and candidate library has its own training-only fixed-recipe comparator. Unseen-recipe training uses 96 candidates and evaluation offers all 120 candidates.

The area-only and delay-only experiments gave mean gains of -0.28% and -0.79%, respectively, with both confidence intervals including zero. Reserving 24 recipes from training gave 0.66% mean gain with interval [-0.28%, 1.56%]. The average benefit in this unseen-recipe setting remains uncertain.

7 Related Work

Mishchenko, Chatterjee, and Brayton introduced scalable AIG rewriting with balancing and established the `resyn2` flow in ABC [2]. Their transformations form the vocabulary used in our recipes.

DRiLLS uses reinforcement learning to construct synthesis sequences and optimizes area under a timing constraint [6]. FlowTune uses a domain-specific multi-armed bandit to explore synthesis flows [7]. These approaches motivate selecting sequences that suit a particular circuit and accounting for the synthesis runs used during exploration.

OpenABC-D provides a large labeled synthesis dataset and supervised prediction tasks involving circuit graphs and recipe encodings [8]. We use a compact tabular representation and focus evaluation on the measured outcomes of selected recipes. Our dataset is generated from the selected benchmark sources using one mapping flow, which keeps the area and delay labels consistent across the pool.

8 Reproducibility and Practical Use

The release contains the final dataset, circuit inventory with source checksums, recipe library, ABC runner, feature extractor, nested evaluation code, fold assignments, out-of-fold predictions, result tables, figures, and a trained model artifact. ABC is pinned to commit `5d64f3b5`; the manifest records its full commit, benchmark commits, library checksum, dataset checksum, sampling seeds, and runtime. Appendix B gives the environment and commands.

We compared all 12,300 overlapping pilot observations with our regenerated measurements. Area, delay, baseline measurements, and all graph and probe features agreed within numerical serialization precision. We independently repeated 24 sampled circuit–recipe pairs, six per source family. Their mapped area and delay reproduced exactly, and ABC established Boolean equivalence between each mapped result and its input circuit. For verification exports we

remove external don't-care annotations, which describe input cases with no required output value, and compare the concrete Boolean networks. All 205 source-file checksums matched the saved inventory.

For a new combinational BLIF, BENCH, or AIGER circuit, the prediction command constructs the graph features, baseline mapping, and probes, then scores the saved recipe library. With a verification budget, it executes the selected shortlist and can write the best mapped circuit as a BLIF netlist. Its predicted score is relative and is used to rank candidates. The verified outputs provide the measured area and delay.

9 Discussion and Conclusion

On the 205-circuit benchmark pool, our adaptive selector gave 0.73% mean improvement over the best fixed training recipe with 95% interval [0.25%, 1.18%]. It closed 9.20% of the aggregate fixed-to-oracle objective gap. The confidence interval lies above zero, supporting an average gain on this circuit pool. Verifying five predicted candidates gave 5.16% mean improvement and 3.95% mean oracle regret. The five-recipe learned shortlist took 3.11 seconds per circuit including feature collection. The global eight-recipe shortlist reached 3.00% regret in 2.88 seconds. For the tested runtime budgets, the global shortlist provided the stronger practical choice. The one-recipe result establishes a small circuit-specific selection benefit; the cost results favor spending this feature-collection overhead on additional candidate executions.

The pool contains 205 independent circuits, and 87% come from MCNC. Family-transfer results therefore carry practical weight when considering a different design population. The labels describe one cell library and one synthesis timing setup. Larger circuits, sequential logic, other cell libraries, and physical implementation require additional measurements.

References

[1] Berkeley Logic Synthesis and Verification Group. ABC: A System for Sequential Synthesis and Verification. `https://github.com/berkeley-abc/abc`.

[2] A. Mishchenko, S. Chatterjee, and R. Brayton. DAG-Aware AIG Rewriting: A Fresh Look at Combinational Logic Synthesis. In *DAC*, pp. 532–535, 2006. `https://doi.org/10.1145/1146909.1147048`.

[3] L. Amarù, P.-E. Gaillardon, and G. De Micheli. The EPFL Combinational Benchmark Suite. In *IWLS*, 2015. Benchmark repository: `https://github.com/lsils/benchmarks`.

[4] ISP RAS. hdl-benchmarks: Collection of Digital Hardware Modules and Projects. `https://github.com/ispras/hdl-benchmarks`.

[5] The OpenROAD Project. NanGate45 Platform and Typical-Corner Open Cell Library. `https://github.com/The-OpenROAD-Project/OpenROAD-flow-scripts/tree/master/flow/platforms/nangate45`.

[6] A. Hosny, S. Hashemi, M. Shalan, and S. Reda. DRiLLS: Deep Reinforcement Learning for Logic Synthesis. In *ASP-DAC*, pp. 581–586, 2020. `https://arxiv.org/abs/1911.04021`.

[7] W. Lau Neto, Y. Li, P.-E. Gaillardon, and C. Yu. FlowTune: End-to-End Automatic Logic Optimization Exploration via Domain-Specific Multi-Armed Bandit. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2022. `https://arxiv.org/abs/2202.07721`.

[8] A. B. Chowdhury, B. Tan, R. Karri, and S. Garg. OpenABC-D: A Large-Scale Dataset for Machine Learning Guided Integrated Circuit Synthesis. arXiv:2110.11292, 2021. `https://arxiv.org/abs/2110.11292`.

A Complete Structural Feature Definitions

Let I be the input count, O the output count, N the AND-node count, and L the graph depth. An edge into an AND node is a *fanin*; edges leaving a signal contribute to its fanout. A node's level is one plus the larger level of its two input signals, with input signals at level zero.

Feature or group	**Definition and interpretation**
Inputs, outputs, AND nodes, depth (4)	I, O, N, L. Interface dimensions, amount of logic, and longest logical path.
Edges (1)	$2N + O$, including each AND node's two inputs and output connections.
Size ratios (4)	N/L, $(2N + O)/(N + I)$, I/N, and O/N. Describe logic density relative to paths and interface size.
Fanout statistics (5)	Mean, maximum, and standard deviation over input and AND nodes; fraction with fanout above one; mean over AND nodes alone. Higher fanout indicates signal reuse or branching.
Complemented-edge fraction (1)	Fraction of AND fanin edges carrying an inverted signal.
Level-width statistics (4)	Maximum and mean number of AND nodes per level, width standard deviation divided by its mean, and maximum width divided by N. Describe how logic is distributed across depth.
Fanin-level imbalance (3)	Mean and maximum absolute level difference between an AND node's two input signals, and the fraction of nodes with a difference of at least two.
XOR-pattern fraction (1)	Fraction of AND nodes matching a local three-node exclusive-OR or exclusive-NOR construction. This is a structural pattern count.
Local reconvergence fraction (1)	Fraction of AND nodes whose two input neighborhoods share a non-constant signal within one predecessor step. This describes nearby paths that diverge and meet again.

The ten log-transformed static columns are input count, output count, AND count, depth, edge count, nodes per depth, maximum fanout, maximum width, mean width, and maximum fanin-level difference. Ratios and fractions retain their original scale.

B Environment and Reproduction

Item	Recorded value
Processor allocation	8 logical CPUs; synthesis sweep: 5 workers
Python	3.12.3
numpy	1.26.4
pandas	3.0.2
scipy	1.17.1
scikit-learn	1.8.0
lightgbm	4.6.0
matplotlib	3.6.3
joblib	1.5.3
ABC commit	`5d64f3b5f3bb9db6f3d77636a65e99ba3ee9d799`
Library SHA-256	`8d540a4d4cf6d09d27c87ad0...`
Cross-validation seeds	0, 1, 2; inner split seed: outer fit seed + 1000
Sampling seeds	Pilot recipes: 7; extension: 17; subset: 23; unseen recipes: 29
Bootstrap	2,000 circuit resamples; seed 2026

From the project root, the main commands are:

```
python3 -m pip install -e .
python3 scripts/fetch_sources.py
make -C tools/abc -j4 ABC_USE_NO_READLINE=1
circuit-recipe sweep --workers 5
python3 scripts/run_experiments.py
python3 scripts/report.py
circuit-recipe train
python3 scripts/build_paper.py
```

The supplied dataset allows the learning and reporting stages to run directly. The source download and sweep commands regenerate the synthesis measurements. Checkpoints preserve completed circuits during a resumed sweep.

For example, to verify three model-ranked recipes on a new circuit and save the best mapped result:

```
circuit-recipe select path/to/circuit.blif --verify 3 \
    --netlist results/optimized.blif
```

The artifact is fitted on all 205 circuits after evaluation. Its training-set fit serves deployment; the reported performance comes from the saved held-out-circuit predictions.